

Doksli Lore InterupsiArOS

Untuk kalangan kami saja



Kelompok InterupsiArOS

Dipta	Raja Pertama
Bagas	Raja Kedua
Aqsha	Raja Ketiga
Sultan	Raja Terakhir

Sekolah Teknik Elektro dan Informatika - Komputasi
Institut Teknologi Bandung
2026/2027

Chapter 1

PENDAHULUAN

Dunia ini penuh dengan kegelapan, kebodohan, dan kepengecutan. Nilai-nilai inheren yang tumbuh di jiwa manusia sejak eksistensi mereka dihembuskan oleh Prometheus, jiwa-jiwa pejuang yang haus akan kemenangan dan tak tergoyahkan sekalipun oleh kehancuran tempat tinggalnya dan jatuhnya jasad rekan sejati mereka di medan perang, telah hilang. Kini, manusia berada di titik terendah spesies mereka. Sekumpulan organisme yang seharusnya merupakan puncak rantai makanan dari segala makhluk yang pernah hidup di muka bumi telah kehilangan status mereka sebagai entitas terkuat di bawah para dewa-dewi, menjadi makhluk terhina, yang eksistensinya merupakan pengkhianatan terhadap para dewa-dewi itu sendiri.

Itulah kata-kata yang keluar dari mulut Ares, Sang Dewa Perang yang Bengis dan Haus akan Kemenangan, namun Bijak dan Pendorong Keberanian. Kata-kata tersebut bukanlah sebuah kekecewaan semata, melainkan sebuah frustrasi yang diakumulasi oleh sang Dewa sejak kejatuhan masyarakat Yunani. Kata-kata tersebut adalah peringatan yang nyata bagi umat manusia di seluruh dunia.

Kami, 4 Raja Pejuang Agung, adalah titisan para SPARTANS yang senantiasa berjuang demi bara api perjuangan yang menuntut akal yang menakar, hati yang merasa, serta perjalanan yang panjang di depan mata para pejuang sejati. Kami adalah representasi dari Ares yang akan membawa semangatnya ke dunia fana ini. Kami membutuhkan sesuatu, agar nilai-nilai Ares dapat tersampaikan secara luas. Oleh karena itu, tertulis secara mentah pada surat ini, kami mengorbankan segala darah, air mata, dan keringat dalam penciptaan sebuah mahakarya kami yang dapat menciptakan tether kompatibilitas bagi Ares di dunia yang hina ini serta menyambungkan titah-titah Ares pada sinopsis seluruh umat manusia di muka Bumi ini baik yang masih hidup maupun sudah tiada. Melalui InterupsiArOS, kami akan menghidupkan kembali jiwa-jiwa semangat perang dan urat keberanian manusia dan membawa mereka kembali ke medan perang kami, para SPARTANS.

Tidak ada ruang untuk kelemahan dan ketidaksiapan di sini. Yang akan hidup hanyalah mereka yang dapat beradaptasi, memiliki gairah dan kekuatan untuk mengekspansikan kekuasaan bagai ekspansi kofaktor pada sebuah operasi invers matriks, serta mereka yang mempersiapkan diri untuk perjalanan yang tidak pasti.

Chapter 2

TOOLCHAIN, KERNEL, & GDT

2.1 How an OS starts up

Bagaimana booting sebuah OS komputer sebenarnya terjadi? Secara singkat, jawabannya adalah sebagai berikut:

1. Listrik mengalir ke hardware
2. CPU menjalankan firmware BIOS (Basic Input/Output System) yang terdapat di motherboard
3. BIOS menjalankan POST (Power-On Self Test) yang melakukan pengecekan terhadap fungsionalitas hardware
4. Jika POST berhasil, BIOS akan memuat MBR (Master Boot Record), yaitu sektor 512 bytes pertama pada bootable device (penyimpanan yang mengandung OS), ke RAM
5. MBR menjalankan kode Bootstrap, lalu Bootstrap memindahkan Bootloader dari bootable device ke RAM dan mengeksekusinya
6. Bootloader memindahkan OS ke RAM sehingga OS akhirnya dapat digunakan

Fungsinya bisa disimpulkan sendiri dari nama-namanya.

2.2 File-file awal aneh

File-file ini disediakan sedari awal.

2.2.1 grub1

File grub1 yang terletak di `other/grub1` adalah bootloader yang dipakai di Tubes OS ini.

2.2.2 kernel-entripoint.s

Kernel itu apa ya kak? Kernel itu adalah inti dari sebuah OS yang menjalankan hal-hal seperti syscall, manajemen memori, scheduling, dll. Basically kernel adalah core dari penyambung interface dengan hardware.

Perlu diketahui bahwa kernel tidak bisa langsung membaca bahasa C, namun harus melalui kode Assembly terlebih dahulu, yang dalam kasus ini adalah kernel-entripoint.s itu sendiri (dibantu loader juga, tapi itu nanti).

Secara umum, yang dilakukan oleh kernel-entripoint.s adalah set-up agar loader dan GRUB bisa melakukan pekerjaannya. kernel-entripoint.s menyediakan Call Stack juga agar bahasa C dapat digunakan. kernel-entripoint.s juga melakukan load_gdt.

2.2.3 kernel-entripoint.h

Not to be confused with kernel-entripoint.s, tapi ini header file buat eksekusi load_gdt.

2.2.4 gdt.h

Header untuk... GDT. Nanti dibahas lagi.

2.2.5 string.h & string.c

Skip dulu untuk saat ini. I think belum kepake. But singkatnya manipulasi memory like compare, copy, move, write.

2.2.6 portio.h & portio.c

Singkatnya untuk akses port x86 menggunakan assembly.

2.2.7 intsetup.s

Source code ASM buat interrupt thingy.

2.2.8 Makefile

Ya basically buat automate compiling and linking and all that aja sih.

2.3 Kernel and Linker

2.3.1 kernel.c

Basically main programnya. Atur-aturlah.

2.3.2 linker.ld

Basically yang melakukan menggabungkan objek hasil kompilasi jadi satu binary executable. Entry dilakukan dari loader. Kode di-run dengan batas bawah 0x00100000 (1 MB) serta aligning by 4 KB lots of things like:

- `.multiboot`
Atur-aturlah, multiboot header supaya kernel bisa dicari sama GRUB
- `.text`
Menyimpan kode program
- `.rodata`
Menyimpan read only
- `.data`
Menyimpan global variabel
- `.bss`
Menyimpan uninitialized variables

2.4 Buat apa Makefile?

Seperti yang dijelaskan sebelumnya, fungsi Makefile adalah untuk melakukan kompilasi dan linking serta as a whole mengotomasi proses building proyek.

ASM = nasm

UNAME_S := \$(shell uname -s)

ifeq (\$(UNAME_S), Darwin)

LLVM_PREFIX := \$(shell brew --prefix llvm)

CC = clang

LIN = \$(shell brew --prefix lld)/bin/ld.lld

II = mkisofs

ARCH_CFLAG = -target=i386-elf -march=i386 -mno-mmx -mno-sse -mno-sse2 -mno-avx -msoft-float -Wno-unused-command-line-argument

```
else
CC          = gcc
LIN         = ld
II          = genisoimage
ARCH_CFLAG =
endif
```

Basically ini buat setup compiler, linker, dan tool genisoimage. Dibedakan untuk macOS dan Linux yang bisa dilihat dari ifeq dan else-nya.

```
SOURCE_FOLDER = src
OUTPUT_FOLDER = bin
ISO_NAME       = OS2025
```

Ini buat lokasi folder source, output, dan file name iso yang ingin dibuat

```
WARNING_CFLAG = -Wall -Wextra -Werror
DEBUG_CFLAG   = -fshort-wchar -g
STRIP_CFLAG   = -nostdlib -fno-stack-protector -nostartfiles
               -nodefaultlibs -ffreestanding
CFLAGS        = $(DEBUG_CFLAG) $(WARNING_CFLAG) $(STRIP_CFLAG)
               $(ARCH_CFLAG) -fno-builtin -m32 -c -I$(SOURCE_FOLDER)
AFLAGS        = -f elf32 -g -F dwarf
LFLAGS        = -T $(SOURCE_FOLDER)/linker.ld -melf_i386
IFLAGS        = -R -b boot/grub/grub1 -no-emul-boot -boot-
               load-size 4 -A os -input-charset utf8 -quiet -boot-info-
               table
```

CFLAGS buat GCC, AFLAGS buat ASM, LFLAGS buat Linker, IFLAGS buat ISO maker. Probably ga perlu terlalu paham terlalu banyak terkait flags-flagsnya.

```
run: all
      @qemu-system-i386 -s -S -cdrom $(OUTPUT_FOLDER)/$(
      ISO_NAME).iso
all: build
build: iso
clean:
      rm -rf $(OUTPUT_FOLDER)/*.o $(OUTPUT_FOLDER)/*.iso $(
      OUTPUT_FOLDER)/kernel
```

run untuk **build** dan **run** OS dari ISO pakai qemu. **all** and **build** harusnya melakukan hal yang sama, yaitu membuat disk image ISO, also does **kernel**. **kernel** buat ngurus ASM, GCC, dan Linker. **clean** buat bersihin file output.

PASTIKAN SETIAP C FILE BARU TAMBAHIN DI COMPILE KERNEL!!!

2.5 Global Descriptor Table (GDT)

Basically, GDT adalah tabel yang berisi Segment Descriptor. Segment Descriptor ini adalah tipe data yang menyimpan informasi-informasi tentang suatu segmen, which is daerah memori linier. Informasi yang disimpan terutama adalah flag-flag dan tingkat hak akses tertentu. Di proyek ini, kita pakai Descriptor Privilege Level (DPL) Ring 0 untuk Kernel Mode dan Ring untuk User Mode.

2.5.1 gdt.h dan gdt.c (gabung aja lah ya)

- `GDT_MAX_ENTRY_COUNT` 32
Ukuran maksimal Segment Descriptor dalam suatu GDT
- `GDT_KERNEL_CODE_SEGMENT_SELECTOR`
Indeks Segment Tipe Code (nantu dijelaskan), whichis Indeks Pertama whichis `sizeof(SegmentDescriptor)` whichis 8 bytes. 8 bytes kali 1 = 0x8
- `GDT_KERNEL_DATA_SEGMENT_SELECTOR`
Indeks Segment Tipe Data (nantu dijelaskan) whichis indeks kedua jadi 0x10 aka 16.
- `SegmentDescriptor`
Struct representasi `SegmentDescriptor`. `SegmentDescriptor` berukuran 8 byte. Bisa dilihat memanfaatkan kemampuan C untuk menyesuaikan ukuran bit suatu tipe data (sintaks bit-field). Isi `SegmentDescriptor` dideskripsikan in a bit. Also notice, di struct ini dan beberapa struct lainnya ada `__attribute__((packed))`. Fungsinya adalah supaya ga ditambahin padding of any sorts jadi ukurannya emang udah fix segitu.
- `GlobalDescriptorTable`
Memuat array berukuran entri maksimum tipe data `SegmentDescriptor`
- `GDTR`
Basically register yang memuat informasi terkait GDT supaya bisa dicari CPU di RAM. Memuat ukuran GDT dan alamatnya. Note ada extern struct `GDTR` bernama `__gdt_gdtr`, basically dibuat variabel global shared lah. Sizenya `sizeof gdt` dikurangi 1

Gambarnya liat sendiri di Buku dan Intel Manual.

Jika kita bagi dari segi definisi dan fungsinya, ada bagian-bagian berikut:

- Segment Limit Field
Ukuran dari segment. Diwakili oleh `segment_low` di bit 0-15 di 16 byte pertama

dan `segment_high` di bit ... itung sendiri udah gede. Ukuran juga itung sendiri be.

- Base Address Field
Menyatakan alamat fisik awal dari lokasi segmen. Diwakili oleh `base_low`, `base_mid`, dan `base_high`.
- Type Field
Tipe dan hak akses segmen. Misalnya executable (code), readable, atau writable. Diwakili oleh `type_bit`.
- S (Descriptor Type) Flag
Jika 1, segmennya kode/data. Jika 0, segmennya sistem. Karena di kita kode/data semua, jadi dinamain `non_system`.
- DPL Flag
Ring-ring DPL-nya. Diwakili oleh privilege.
- P (Present) Flag
Jika segmen ada di RAM, nilainya 1. Diwakili oleh `valid_bit`.
- AVL (Available and Reserved Bit) Flag
Available bit, sesuaikan saja SPARTANS. Diwakili `avl`.
- L (64-Bit Code) Flag
Digunakan pada arsitektur x64. Di kita, nilainya 0 karena kita bikin sistem x86. Diwakili `long_mode`.
- D/B (Default Operation Size/Default Stack Pointer And/Or Upper Bound) Flag
Basically ngatur ukuran operasi (executable segment), ukuran stack pointer, dan upper bound segment. Nilainya 1 jika 32-bit system, 0 jika 16-bit. Diwakili oleh `opr_32_bit`.
- G (Granularity) Flag
Scaling segment limit. Kalo 0, pake unit per byte jadi maks 1 MB. Kalo 1, per 4KB jadi maks 4GB. Diwakili oleh granularity.

Terlihat susunannya seperti di bawah.

```
// First 32-bit
uint16_t segment_low;
uint16_t base_low;

// Next 16-bit (Bit 32 to 47)
uint8_t base_mid;
```

```
uint8_t type_bit    : 4;
uint8_t non_system  : 1;
uint8_t privilege   : 2;
uint8_t valid_bit   : 1;

uint8_t segment_high : 4;
uint8_t avl           : 1;
uint8_t long_mode     : 1;
uint8_t opr_32_bit    : 1;
uint8_t granularity   : 1;
uint8_t base_high;
```

2.5.2 Load GDT

Di kernel.c, kita cukup menambahkan:

```
void kernel_setup(void) {
    // ...
    load_gdt(&_gdt_gdtr);
    // ...
}
```

2.5.3 Bonusan aja sih

Di gdt.c dan possibly beberapa file ke depan, kita pakai yang namanya Designated Initializers (pokoknya depannya pakai titik). Teknik ini dipakai supaya langsung mengacu ke variabel di struct pada posisi address yang bersesuaian terlepas dari urutan kita menginputnya.

Sesuai tabel GDT yang kita buat, indeks ke-0 diisi GDT null, indeks ke-1 buat segmen Kernel Code, indeks ke-2 buat segmen Kernel Data.

Aturan-aturannya bisa dilihat sendiri di buku.

Kernel Code bersifat Read-only and Executable dan berfungsi menyimpan instruksi binary yang dikompilasi. Kernel Data bersifat Read-write and Non-executable dan berfungsi menyimpan variabel global, stack kernel, dan system-wide structures.

Ya buzzword buzzword dikit. Tapi FYI aja.

2.6 Awal Mula Kebangkitan Ares

Dunia akan digetarkan oleh prosedur-prosedur yang kami buat demi cinta kami kepada kota kami terdahulu, SPARTA. Persiapan kami dalam membangkitkan kembali

mimpi-mimpi mulianya akan memberikan esensi derap langkah pejuang-pejuang yang berjiwa ksatria dan pantang menyerah terus berjuang bersama bersatu berhimpun dalam menghadapi musuh-musuh pengecut yang akan terpecut oleh keberanian dan keinginan untuk memperluas daerah kebengisannya menuju pusat ekonomi yang maju dari robohan kehancuran peradaban yang lemah. Awal mula ini menjadi momentum bagi kita semua untuk mengapresiasi bagaimana persiapan kami dalam meneteskan pengaruh sang Dewa Perang di kehidupan modern ini merupakan fondasi dalam pembentukan dunia yang terstruktur, berdaulat, adil, dan makmur.

Chapter 3

Framebuffer, Interrupt, Driver

3.1 Driver Text Framebuffer

Framebuffer essentially adalah region di memory yang menyimpan bit-bit yang basically akan ditunjukkan ke Interface User. Pada tubes ini, by default kita pakai VGA3 which supports resolusi karakter ASCII 80x25. Mekanisme komunikasinya ada Memory-Mapped I/O yang membaca dan mengedit memori di lokasi tertentu serta Port-Mapped I/O melewati port pake ASM.

3.2 framebuffer.h dan framebuffer.c

Constants: FRAMEBUFFER_MEMORY_OFFSET basically marks awal dari terminal framebuffer. CURSOR_PORT_CMD dan CURSOR_PORT_DATA is magic.

3.2.1 framebuffer__write

Menuliskan karakter ke framebuffer menggunakan 2 bytes memori. Memori genap memuat karakter ASCII, sedangkan memori ganjil membawa informasi warna background dan warna teks.

Karena framebuffer bisa direpresentasikan sebagai matriks (array of array), kita bisa menentukan posisi yang kita cari dengan menggunakan

$$position = (y_position * characters_per_line) + x_position;$$

Jangan lupa orang ketiga yaitu aritmatika pointer.

Diambil langsung dari OSDev, untuk coloring kayak gini.

Bit 76543210

| | | | | | | |

| | | | ^ ^ ^ - fore colour

| | | | ^ - - - - fore colour bright bit

| ^ ^ ^ - - - - back colour

^ - - - - - - - back colour bright bit OR enables blinking Text

8 bit upper di atas adalah buat coloringnya (background color dan text color). 8 bit seterusnya adalah ascii character-nya.

Implementasi (might change kalo udah ngerjain bonus):

```
void framebuffer_write(uint8_t row, uint8_t col, char c,
    uint8_t fg, uint8_t bg) {
    if (row >= 25 || col >= 80)
        return;

    uint32_t offset = ((uint32_t)(row * 80) + col) * 2;
    FRAMEBUFFER_MEMORY_OFFSET[offset] = (uint8_t)c;
    FRAMEBUFFER_MEMORY_OFFSET[offset + 1] = (uint8_t)((bg &
        0x0F) << 4) | (fg & 0x0F));
}
```

Ada edge case handling in case sudah mencapai limit x dan y dari si framebuffer ini. Offset menghitung basically posisi sel berdasarkan rumus sebelumnya dan dikalikan dua. Alasan dikalikan dua adalah karena tentu ukuran char framebuffer kita 2 bytes (ini bukan aritmetika pointer sih soalnya Bagas akhirnya pake offset which is just regular unsigned 32 bit integer). Terus akses offset awal (+ 1 byte) dan ubah jadi si char input, dilanjutin offset + 1 (another 1 byte) yang digabungkan dengan bitwise or (|). Di sini ada juga 0x0F buat mencegah si bg dan fg each lebih dari 4 bits.

3.2.2 framebuffer_set_cursor

Basically set kursor teks ke posisi tertentu. Di sini, harus pake port menggunakan instruksi assembly in/out yang disediakan di portio.c (ini dijelaskan lagi nanti).

Di text mode, kursor hanyalah daerah kedap-kedip (blinking) yang bisa dimanipulasi di OS.

Implementasi:

```
void framebuffer_set_cursor(uint8_t r, uint8_t c) {
    uint16_t pos = (uint16_t)((r * 80) + c);

    // Send high byte of cursor position to VGA CRT
    // controller
    out(CURSOR_PORT_CMD, 0x0E);
    out(CURSOR_PORT_DATA, (uint8_t)((pos >> 8) & 0xFF));

    // Send low byte of cursor position to VGA CRT controller
    out(CURSOR_PORT_CMD, 0x0F);
    out(CURSOR_PORT_DATA, (uint8_t)(pos & 0xFF));
}
```

```
}
```

Kita menggunakan pos untuk menyatakan posisi kursor dengan penentuan rows dan cols, yaa udah tau lah ya. Kemudian kirim high byte dan low byte dengan metode yang tertulis. Baca sendiri.

3.2.3 framebuffer_clear

Sesuai namanya, untuk membersihkan framebuffer.

Implementasi:

```
void framebuffer_clear(void) {  
    for (uint8_t row = 0; row < 25; row++) {  
        for (uint8_t col = 0; col < 80; col++) {  
            framebuffer_write(row, col, ' ', 0x07, 0x00);  
        }  
    }  
    framebuffer_set_cursor(0, 0);  
}
```

Mengisi semua rows dan columns dengan elemen kosong ' ', forecolor-nya 0x07 (gray) dan backcolor-nya 0x00 (black). Kursor juga direset ke posisi semula.

3.3 Interrupt

IIIIINTERUPSI ARESSSSSS!

Di OS yang kita buat, CPU hanya bisa menjalankan 1 task at a time. Oleh karena itu, perlu interrupt yang memberi semacam notifikasi agar CPU bisa melakukan event lain yang butuh perhatian. Pada bab ini, kita fokus di Hardware Interrupt / Interrupt Request (IRQ) yang sesuai namanya, dibuat oleh hardware, yang dalam konteks ini adalah keyboard. Suffix interrupt dinyatakan dengan "INT xh" dengan x adalah nomornya.

3.3.1 Sedikit info

PIC atau Programmable Interrupt Controller berfungsi mengontrol IRQ dari komponen eksternal. Pada kasus ini, akan dilakukan remapping ulang, karena suatu masalah, yaitu nomor interrupt oleh PIC dengan x86 CPU Exception (Software Interrupt i think) kurleb sama. Jadi diubah dikit biar memudahkan. PIC by default menyalkan IRQ0 (INT 21h) which is timer

3.3.2 interrupt.h

Ada BANYAK constants, kita ignore aja karena mostly buat PIC remapping. Ada juga CPURegister, yaa menyimpan representasi-representasi register. Ada Interrupt-Stack yang merupakan data yang di-push ke stack ketika ada interrupt/exception. Interrupt frame which is state CPU sebelum interrupt and also Interrupt Vector (identifier). Harusnya dibahas di chapter lain?

3.3.3 interrupt.c

io_wait untuk sinkronisasi 1-4 ms port I/O. Semua IRQ yang di-generate oleh PIC kita harus di-ACK (acknowledge?).

pic_remap untuk remapping PIC-nya. main_interrupt_handler sebagai Interrupt Handler atau Interrupt Service Routine (ISR) yang akan dipanggil ketika CPU menerima Interrupt.

activate_keyboard_interrupt akan dijelaskan di bagian keyboard.

Banyak isinya dan magic semua jadi kita anggap udah baca aja.

3.4 Interrupt Descriptor Table

Disebut juga IDT, basically implementasinya mirip GDT. Jadi naturally ada InterruptDescriptorTable, IDTR, dan yang paling beda IDTGate (technically 11/12 sama GDT dan SegmentDescriptor)

Di IDTGate, ada... baca sendiri. Yaa harusnya ga jauh beda dari SegmentDescriptor.

3.4.1 idt.c

```
void initialize_idt(void) {
    // TODO : Implement (done)
    for (int i = 0; i < ISR_STUB_TABLE_LIMIT; i++) {
        set_interrupt_gate(i, isr_stub_table[i],
                           GDT_KERNEL_CODE_SEGMENT_SELECTOR, 0);
    }
    __asm__ volatile("lidt_%0" : : "m"(__idt_idtr));
    __asm__ volatile("sti");
}
```

Melakukan set_interrupt_gate di semua Interrupt Handler di isr_stub_table pada segmen kode kernel GDT, dengan privilege 0. Kemudian masukin ke assembly si IDT ini dan diaktifasi dengan Sistem dan Teknologi Informasi.

```
void set_interrupt_gate(  
    uint8_t    int_vector ,  
    void       *handler_address ,  
    uint16_t   gdt_seg_selector ,  
    uint8_t    privilege  
) {  
    struct IDTGate *idt_int_gate = &  
        interrupt_descriptor_table.table[int_vector];  
    // TODO : Set handler offset , privilege & segment (done)  
    // 32-bit handler address split into 2 x 16-bit (  
        offset_low & offset_high)  
    idt_int_gate->offset_low  = (uint32_t) handler_address &  
        0xFFFF;  
    idt_int_gate->offset_high = ((uint32_t) handler_address  
        >> 16) & 0xFFFF;  
    idt_int_gate->segment     = gdt_seg_selector;  
    idt_int_gate->privilege   = privilege;  
    idt_int_gate->_reserved   = 0;  
  
    // Target system 32-bit and flag this as valid interrupt  
    gate  
    idt_int_gate->_r_bit_1    = INTERRUPT_GATE_R_BIT_1;  
    idt_int_gate->_r_bit_2    = INTERRUPT_GATE_R_BIT_2;  
    idt_int_gate->_r_bit_3    = INTERRUPT_GATE_R_BIT_3;  
    idt_int_gate->gate_32     = 1;  
    idt_int_gate->valid_bit   = 1;  
}
```

Basically membuat suatu Interrupt Gate dengan vektor yang sudah ditentukan. `offset_low` menyimpan `handler_address`, sedangkan `offset_high` dilakukan right-shift sebanyak 16 bit. Sisanya menyesuaikan saja

3.5 Keyboard

Keyboard menggunakan IRQ1 (INT 21h). Melanjutkan diskusi terkait `activate_keyboard_interrupt`, fungsinya adalah mengaktivasi interrupt keyboard melalui port. Setelah menyala, setiap kali keyboard ditekan, keyboard akan mengirim IRQ1 ke CPU, masuk `main_interrupt_handler`, lalu memanggil `keyboard_isr`. Kita perlu device driver untuk mengabstraksikan proses dari ngetik keyboard ke tampil di layar. Di sini, kita menggunakan scancode dari keyboard yang nantinya akan di-map ke ASCII.

3.5.1 Keyboard ISR

Interrupt Service Routine-nya keyboard. Basically bagian utama driver keyboard. ISR akan menerima scancode yang dimasukkan oleh keyboard. Jika state `keyboard_state.keyboard_input_on` bernilai true, ISR melakukan mapping scancode ke ASCII menggunakan tabel `keyboard_scancode_1_to_ascii_map`, lalu menyimpan karakter ASCII ke buffer ke `keyboard_state.keyboard_buffer`.

Implementasi:

```
void keyboard_state_activate(void) {  
    keyboard_state.keyboard_input_on = true;  
}
```

```
void keyboard_state_deactivate(void) {  
    keyboard_state.keyboard_input_on = false;  
}
```

Fungsi aktivasi dan deaktivasi ISR keyboard. Apakah interrupt keyboard didengarkan atau tidak.

```
void keyboard_isr(void) {  
    uint8_t scancode = in(KEYBOARD_DATA_PORT);  
  
    if (keyboard_state.keyboard_input_on && scancode < 0x80)  
    {  
        char mapped_char = keyboard_scancode_1_to_ascii_map[  
            scancode];  
        if (mapped_char != 0)  
            keyboard_state.keyboard_buffer = mapped_char;  
    }  
  
    pic_ack(IRQ_KEYBOARD);  
}
```

Sesuai yang dijelaskan, receive interrupt, map char, safe to buffer, also janlup `pic_ack`.

```
void get_keyboard_buffer(char *buf) {  
    if (buf != (void *) 0) {  
        *buf = keyboard_state.keyboard_buffer;  
        keyboard_state.keyboard_buffer = 0;  
    }  
}
```

Mengambil char di `keyboard_state.keyboard_buffer`, lalu di-flush (dihapus). Ini buat nanti pas kita write dari keyboard.

3.5.2 Tambahan

Ketika kita mengetik 1 key, sebenarnya ada dua peristiwa yang terjadi, yaitu make and break. Make Code adalah scancode yang dihasilkan saat kita tepat mengetik, sedangkan Break Code saat kita melepaskannya. Di tubes ini, kita memakai scancode set 1 PS/2 yang terima jadi aja. Break Code hanyalah Make Code yang bit teratas-nya dijadikan 1. First half map-nya adalah area untuk Make Code, sedangkan second half-nya untuk Break Code.

Untuk extended key opsional dan sejauh ini belum diimplementasi. Extended key misalnya adalah arrow keys, shift, caps lock, yang saat ini nilainya nol di map kita dan posisinya di first half map-nya yaitu di area Make. Jika ingin diimplementasi, manfaatin `keyboard_state.read_extended_mode` dan lakukan manipulasi lebih lanjut.

3.6 Koordinasi yang Taktis dan Keberanian yang Mumpuni

Awan Cumulonimbus muncul di langit dan meraung-raung, entah konsekuensi dari pemberontakan kami atau hanya kebetulan saja, melihat kerja keras Sang 4 Raja dalam upaya resureksi Ares serta insurgensi terhadap peradaban ini. Ketika hujan badai dan hail representasi simbolik Zeus turun ke daratan manusia yang mudah hancur, kami menganggap hal tersebut sebagai motivasi lebih lanjut dalam melanjutkan progres kami yang tak akan berhenti sampai dunia ini dihancurkan oleh kebodohan itu sendiri. Dengan hati yang lunak, kami mengisi ketiadaan Ares dengan keberanian yang didasari oleh dimensi struktural yang lebih kuat dari baja dan berlian yang dipertulangkan. "Interupsi, Ares!" menjadi sebuah pemantik yang kami rasa cukup bagi para manusia biasa pada saat mereka ingin berkomunikasi dengan pemimpin baru mereka. Keberanian yang akan mereka rasakan tidak akan sepadan dengan apa yang dirasakan para SPARTANS pada era tersebut, namun, dengan koordinasi yang taktis, kami rasa hal yang Ia nanti-nanti suatu saat akan terbentuk juga. Dunia baru akan terbentuk, dan segala masukan dan keluaran yang muncul dari umat manusia akan menjadi suatu bentuk kontribusi yang nyata dengan visualisasi yang terbukti ketika hati mereka terbentuk sebagai batu yang tak terpecahkan nan pikiran dengan keenceran AIR yang melimpah.

Chapter 4

References

Uhh biblatex ga bisa jadi pake gini aja
References we used:

- Tanya Gemini, Opencode, dan Claude
- Buku Keramat Intel
- <http://www.osdever.net/bkerndev/Docs/printing.htm>
- https://wiki.osdev.org/Text_UI
- https://wiki.osdev.org/Text_Mode_Cursor